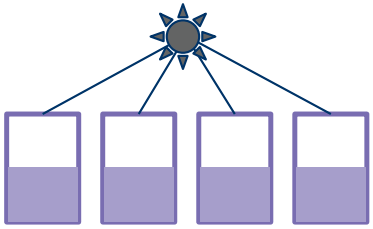
CompSci 390-03-26S

Distributed and Networked Systems

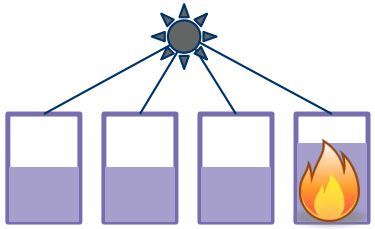
Transactions and Logs

The story so far: servers, services, and state

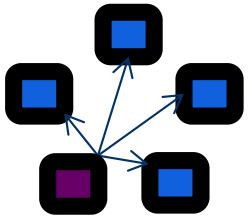
- **Server** processes handle requests from clients
- Requests are RPCs: call method of service API
- Execute API **operation**→transition server **state**
- **Services**: groups of servers coordinate to serve a server API and maintain its state



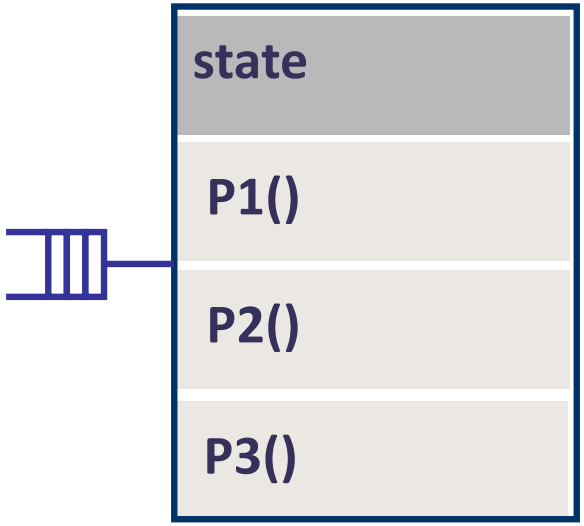
Balanced (L4)



Sharded



Replicated

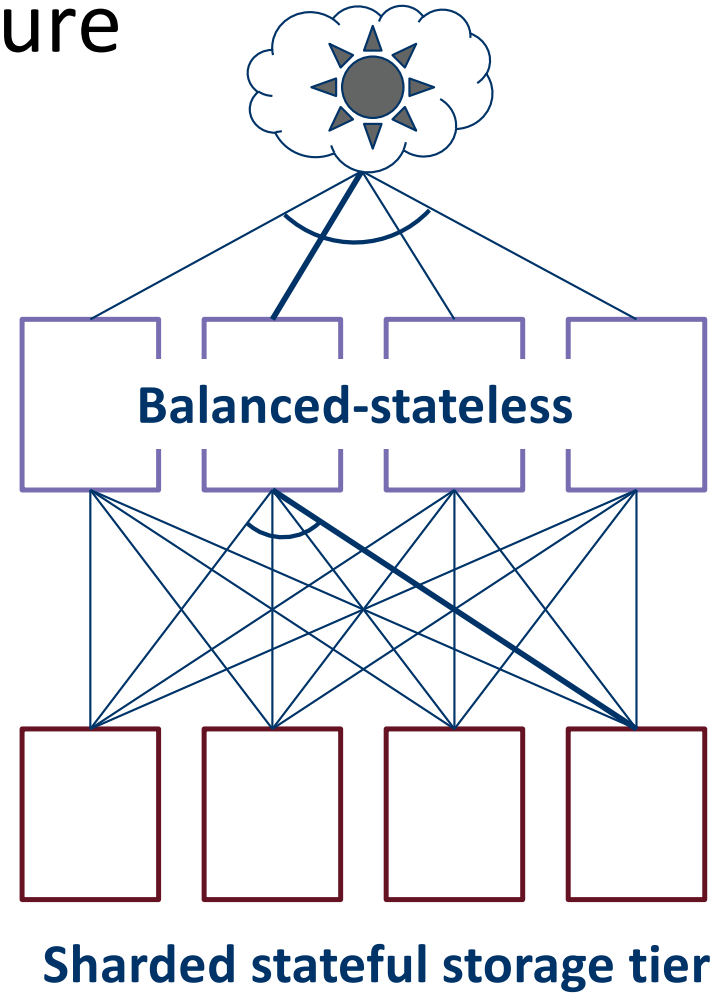


Server module

Typical multi-tier service structure

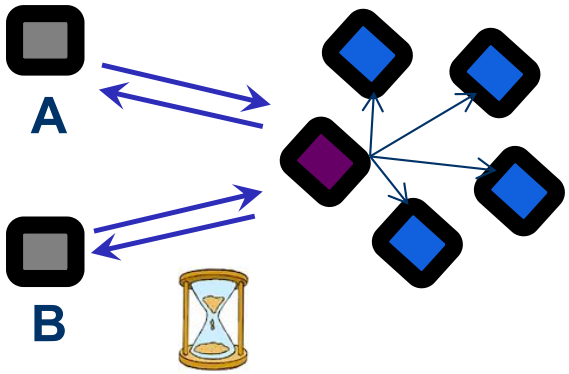
- Front-end tier: "**stateless**"
- **Stateful** storage tier with shards
- Cloud routing to front-end by VIP
- Sharded storage back-ends
- Service state is in storage tier
- Front-ends come and go...

There are lots of degrees of freedom and lots of complexities, but let's take this as our template for cloud-scale services.



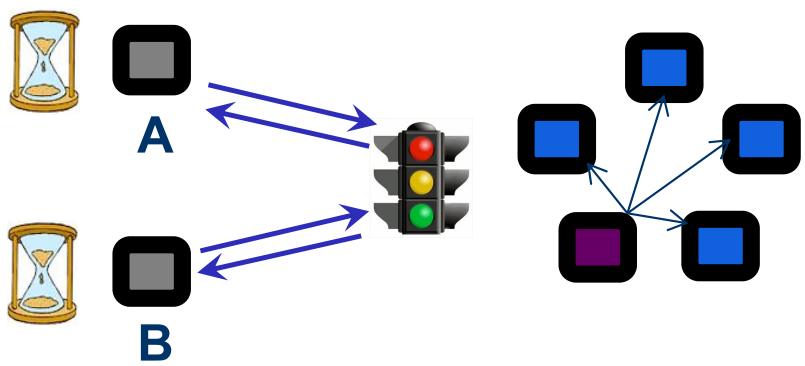
Thinking about service state

- Let's focus on services with **strong consistency** like "Zooky".
- We want a unified timeline for operations, and all clients see the same timeline. This is called **linearizable**.
  - Session leases enable consistent caching and notifications (watches) for linearizable services in presence of failures.
- Let's focus on the state for such services, in the storage tier.
  - Forward tiers are caching clients of storage.
- How to manage state in the storage tier?
- How to ensure it is **consistent** and **available**?



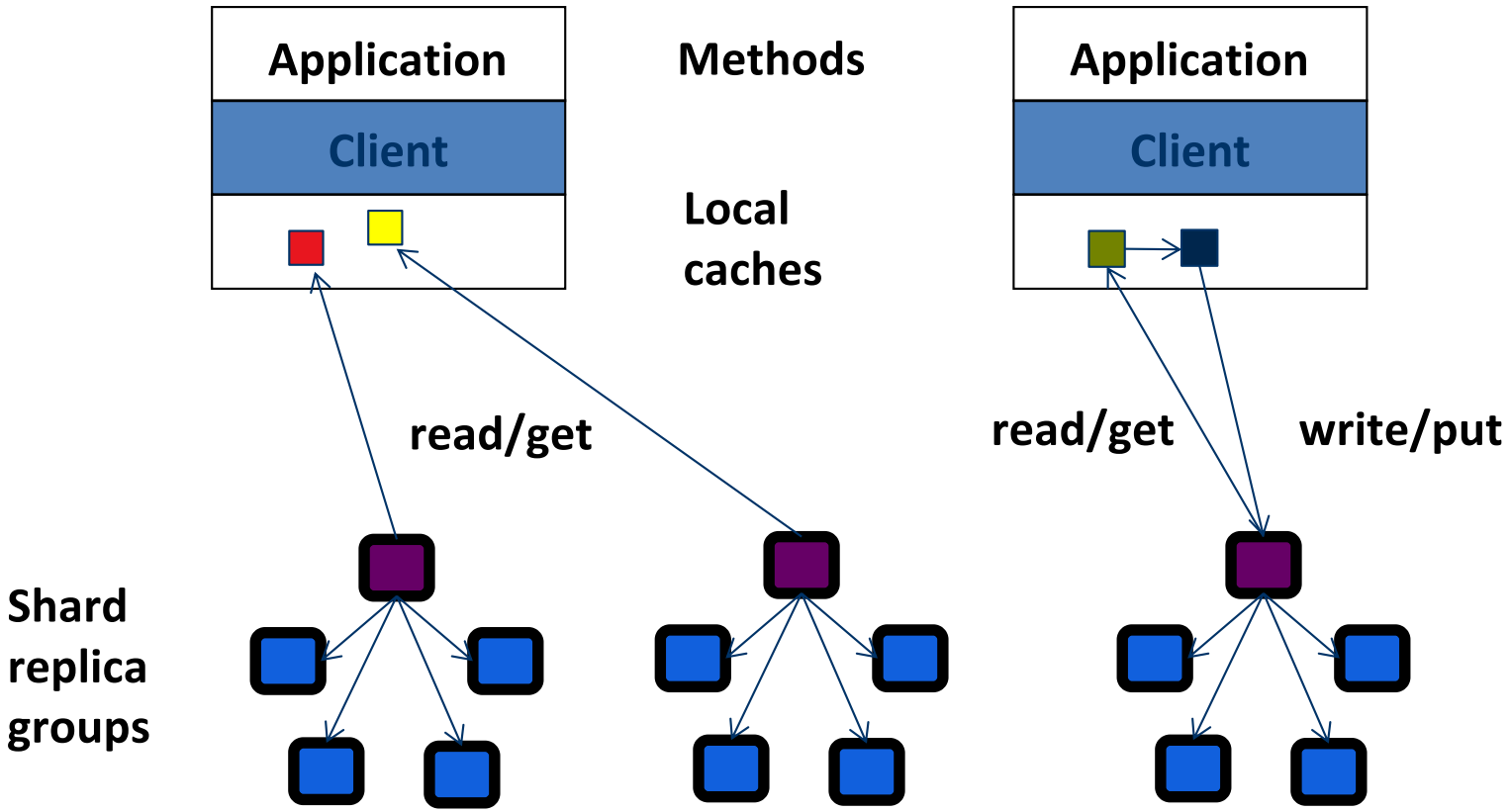
"Zooky" (coordination service)

- Zooky has an API for operating on a set of named **nodes** (small files).
- Create/read/update/delete (CRUD) of nodes in hierarchical namespace
- Watch: clients may request notifications of changes to a node.
- **Consistent**: all clients have the "same" view of Zooky's state, e.g.:
- **Linearizable**: client view reflects a global order of all requests on Zooky.
- Including client-cached reads/writes (unless/until session expires).



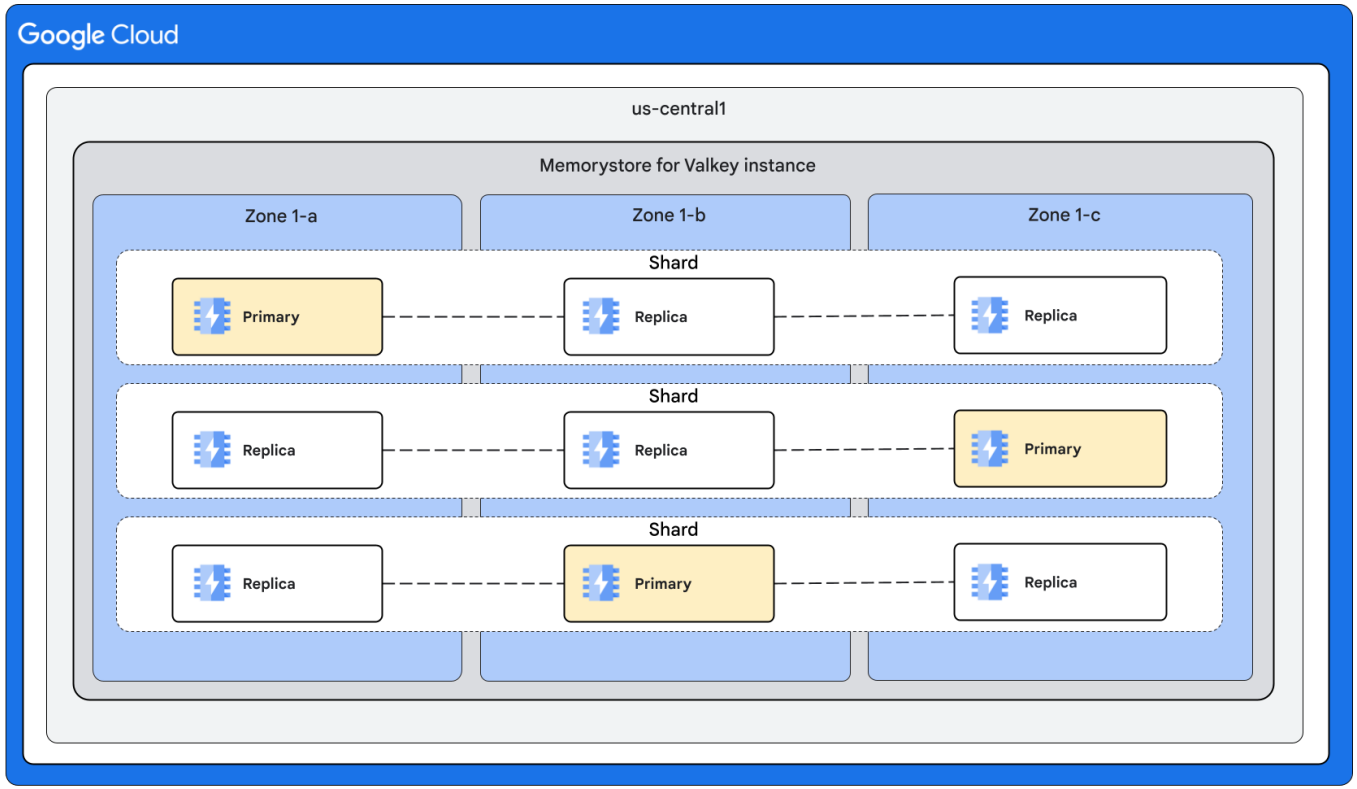
Clients maintain **session leases** with Zooky by regular heartbeat messages. If Zooky does not "hear" a client's heartbeat, then the client's session expires and it loses its lease. Zooky cancels all promises made to that client: stand aside, let's keep it moving.

Sharding and replication together



Example: Google Cloud's Memorystore for Valkey

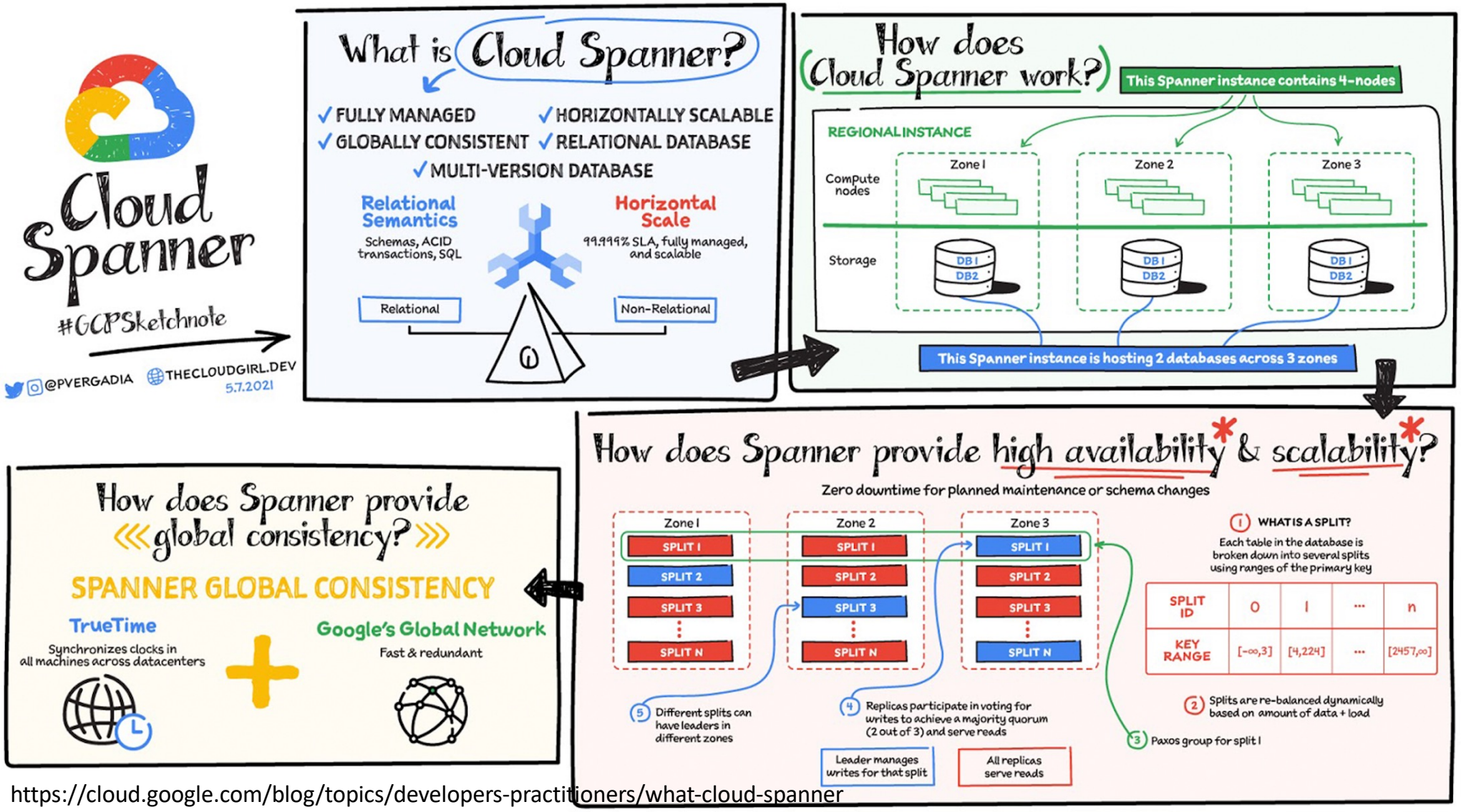
Generic fast in-memory back-end storage tier, offered as a service for cloud customers. Note use of sharding and replication **together**: scale by sharding, protect by replicating.



https://docs.cloud.google.com/memorystore/docs/valkey

Google Spanner

- State of the art for kind of **key-value store** tier we want to consider.
  - Replicated shards
  - Put/get and queries
- Focus: **atomic transactions** for consistent view with complex read/write operations spanning shards.
- All reads/writes have timestamps and are grouped as **transactions** ordered in a real-time sequence.
- Lots of details we gloss over: tables, indexes, range queries over sorted keys, sharding by splits.
- Back-end storage: Colossus, tables: BigTable



https://cloud.google.com/blog/topics/developers-practitioners/what-cloud-spanner

Google Gemini crows about Google Spanner

Roughly **60% of Fortune 500** companies use at least one Google Cloud service, with Spanner being the primary choice for their global transactional workloads (e.g., Goldman Sachs, Walmart).

Ranked **#1** in Gartner's 2025 "Critical Capabilities for Operational Databases" for **Lightweight Transactions**, beating out AWS Aurora and Azure SQL.

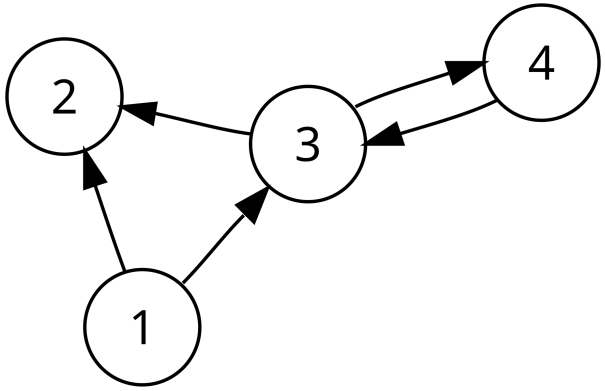
As of 2026, Spanner handles over **6 billion queries per second** at peak and manages more than **17 exabytes** of data globally.

A 2025 Forrester study found that enterprise organizations migrating to Spanner saw a **132% ROI** over three years.

Reported by Gemini on 2/19/26

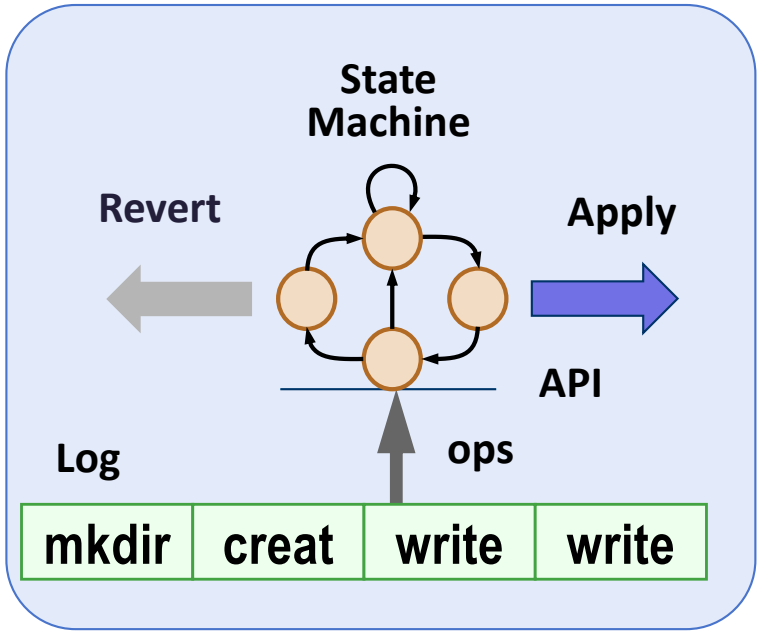
Software as state machines

- Processes have finite **state**.
  - A finite number of variables/objects
  - A finite number of possible values for each
  - Mash into one blob: finite set of possible values
- The **state** of a process is the value of its data "blob".
- Operations (calls to its API) transition the process from one state to another. We can represent it as:
- A **finite state machine:** a directed graph that captures all possible states and transitions.
  - Vertices: possible states
  - Edges: possible transitions between states



Linearizable services as state machines

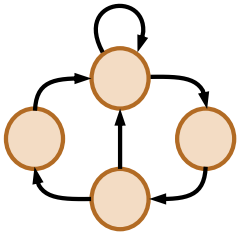
- Any **deterministic** service, E.g., Zooky
- Keep a linear sequence of operations in a **sequential log**.
  - New ops at the **head** of the log
  - Old ops at the **tail** of the log
  - Play/replay the log to roll state forward and "rewind" to roll back (revert).
- Failure? **Recover** state (replay log from some previous state **snapshot**) and restart.
- Recovered state reflects exactly a prefix of ops up to a **recovery point** in the log.



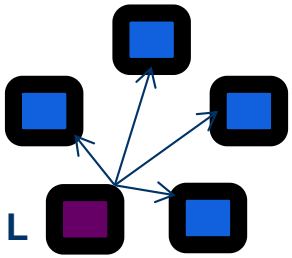
This figure is from the Raft authors

State Machine Replication (SMR/RSM)

- N copies (replicas) of a **deterministic** server.
- Replicas run the same app program P with the same API. It doesn't matter what P is, if it is deterministic.
  - P is a **state machine**: function {state, action}→state

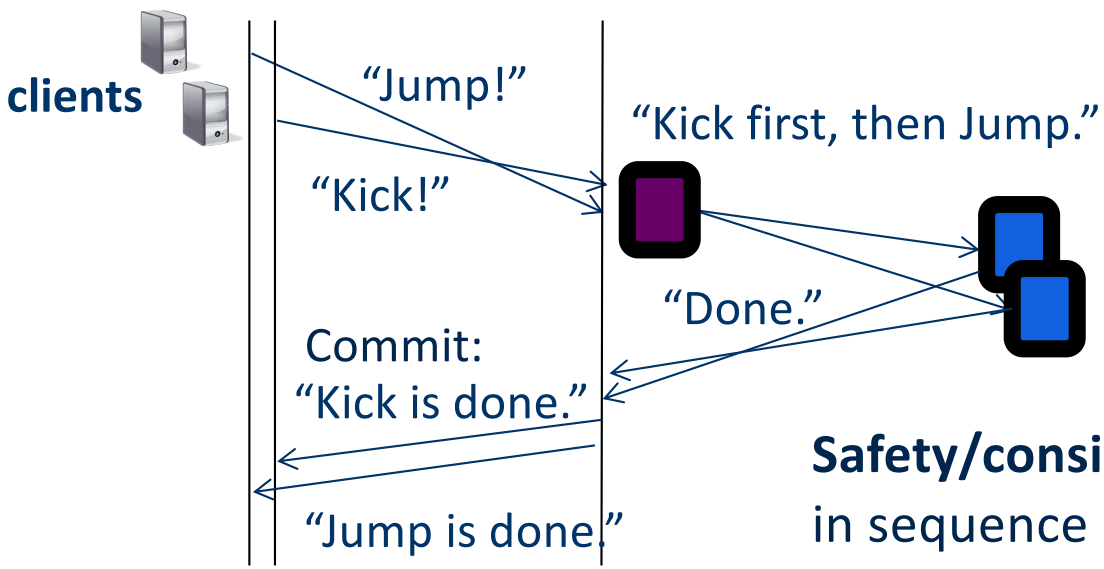


- Each replica keeps its own copy of internal app state.
- Suppose they agree on a sequence (or log) of actions.
  - Ordered list of (requests, commands, ops, values). Linearizable.
- Then they converge.
  - Use any up-to-date replica
  - Or use leader/primary with fail-over
  - Designate stable leader



Primary/backup SMR: basic idea

- Clients send requests to one server (**primary** or **leader**).
- Primary sequences the requests/ops/actions and pushes them to **backup** replicas (**followers**) before responding to clients.



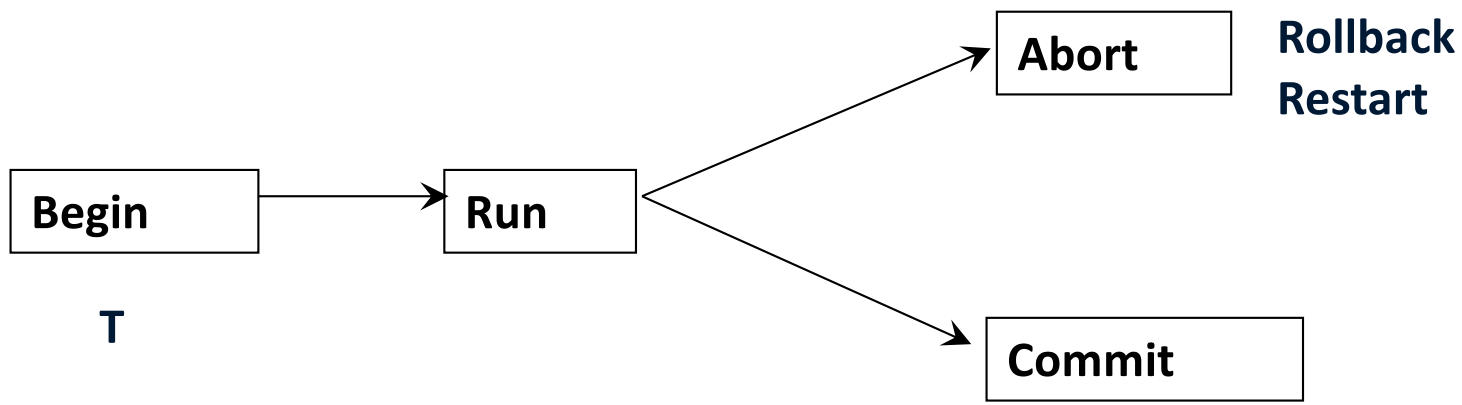
Safety/consistency: ops commit in sequence and committed ops never revert.

What about concurrency?

- Our simplified view of linearizable services involves actions/ops/requests executing **serially**, in sequence.
- But real servers/services involve high degrees of **concurrency**.
- Each action involves a sequence of reads/writes on different parts of the state, for a complex state transition.
- How to interleave/schedule these actions so that they benefit from parallelism but their result **appears** serial?
- How to ensure that failures don't interrupt actions "in the middle" and leave them "partly done".
- **Atomic transactions**: an abstraction to achieve these goals.

Transaction states

- Each transaction T runs as a thread that transitions the data from one consistent/meaningful state to another.
- The program or system may choose to abort (or **restart**) T.
  - e.g., due to an error, deadlock, conflict, failure…
  - Abort "undoes" (rolls back) any updates initiated by T.
- Else T ends with a **commit**.



ACID transactions

A **transaction** executes a sequence of reads/writes on different data items.

```
BEGIN T1
  read X
  read Y
  …
  write X
END
```

```
BEGIN T2
  read X
  write Y
  …
  write X
END
```

Atomic
Consistent
Independent
Durable

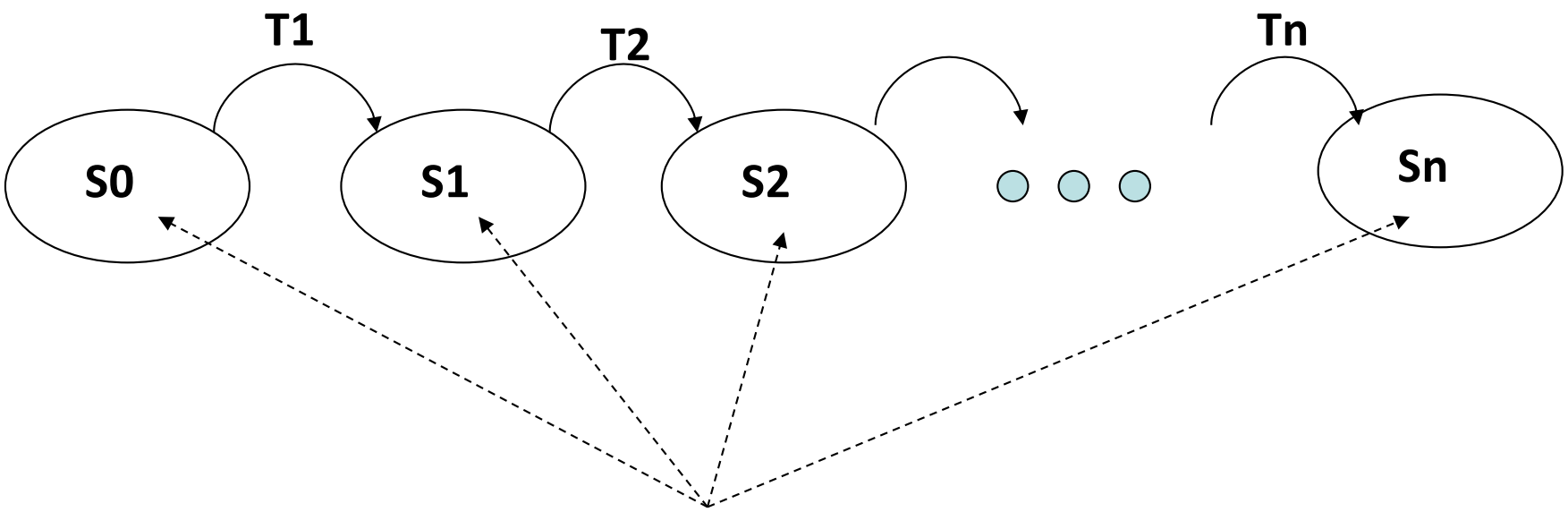
Transactions **appear** to commit (complete) **atomically** in a serial order.

Example transaction

```
BEGIN TRANSACTION;

// Omitted: confirm product is in stock and bank account has adequate funds

UPDATE products
SET stock_quantity = stock_quantity - 1
WHERE product_id = 42;

INSERT INTO orders (customer_id, order_date, total_amount)
VALUES (101, GETDATE(), 59.99);

INSERT INTO bank_transactions (account_id, transaction_type, amount, order_id)
VALUES (5005, 'debit', 59.99, (SELECT LAST_INSERT_ID()));

COMMIT;
CATCH (error)
    ROLLBACK;
END;
```

Serializability and data states

- Data is in a "good" (semantically consistent) state "between" transactions.
  - Transaction bodies must be coded correctly to ensure this!
- **Serializability**: Transactions observe the effects of their predecessors, and not of their successors in a hypothetical (equivalent) **serial schedule**.



A-C-I-D

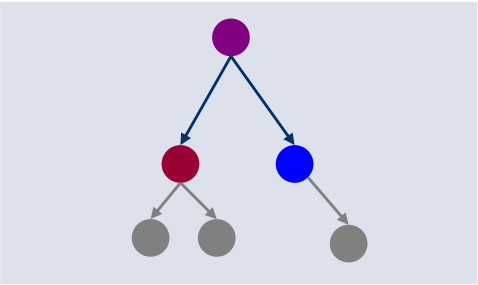
- **Atomic**: a transaction T is an atomic/indivisible unit. "All or nothing"
  - T either commits or aborts
  - **Commit**: all of T's ops complete, and visible to subsequent transactions
  - **Abort**: None of them complete, "like it never happened".
- **Consistent**: "Each T does something reasonable."
- **Independent**/Isolated: "Transactions don't interfere with one another"
  - All updates by T1 are either entirely visible to T2, or are not visible at all.
- **Durable**: if a transaction T commits, then its effects survive a(ny) subsequent failure. "Never lost"

Transactions: ACID Properties

- ***Atomicity***. The system guarantees atomicity using **logging**, shadowing, and **two-phase commit** (2PC).
- ***Consistency***. The application guarantees consistency by being correct and correctly marking transaction boundaries.
- ***Independence/Isolation***. Guaranteed through **two-phase locking** (2PL) or **optimistic concurrency control** (OCC).
- ***Durability***. The system guarantees durability by writing committed updates to stable/recoverable storage (e.g., logging, replication, or both).

Recoverable in-memory data with a simple log

Your data structures in memory

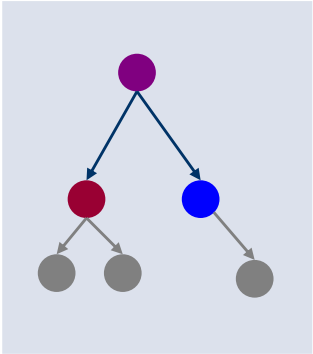


Your program executes transactions that read or update the data structures in memory.

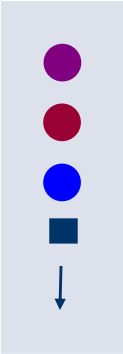
Volatile

Persistent (e.g., "disk")

Save a checkpoint or snapshot of the entire structure periodically. **Must be atomic**.



snapshot

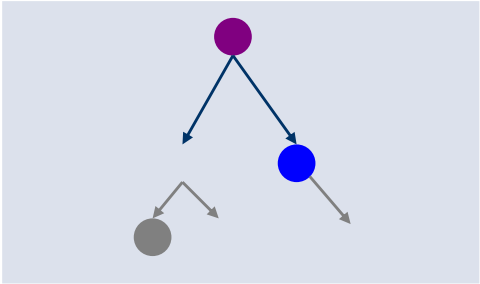


log

Log operation steps as they occur. Discard after each snapshot (log truncation).

A simple example of logging

Q. What if the structure is too large to fit in memory? Like a file system or database?

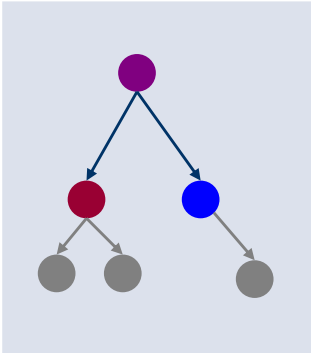


A: keep a partial copy in memory. Update there, and push writes to full on-disk copy ("home") later.

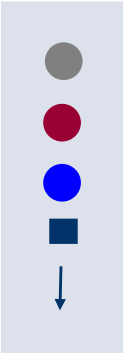
Volatile

Persistent (e.g., "disk")

Maintain full state on disk in the usual way; supplement with a log file.



home

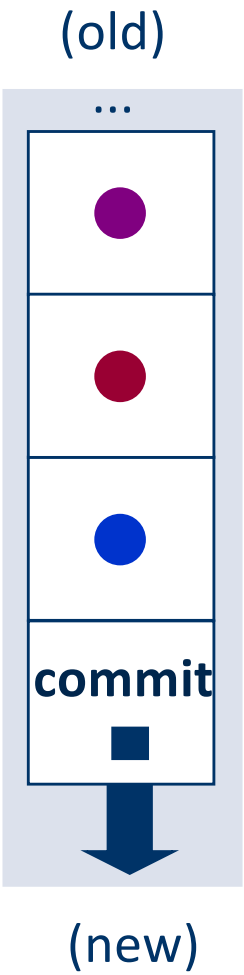


log

Log operation steps by appending to a log file on disk. Make sure the commit is logged **before** pushing any writes to home—**WAL**.

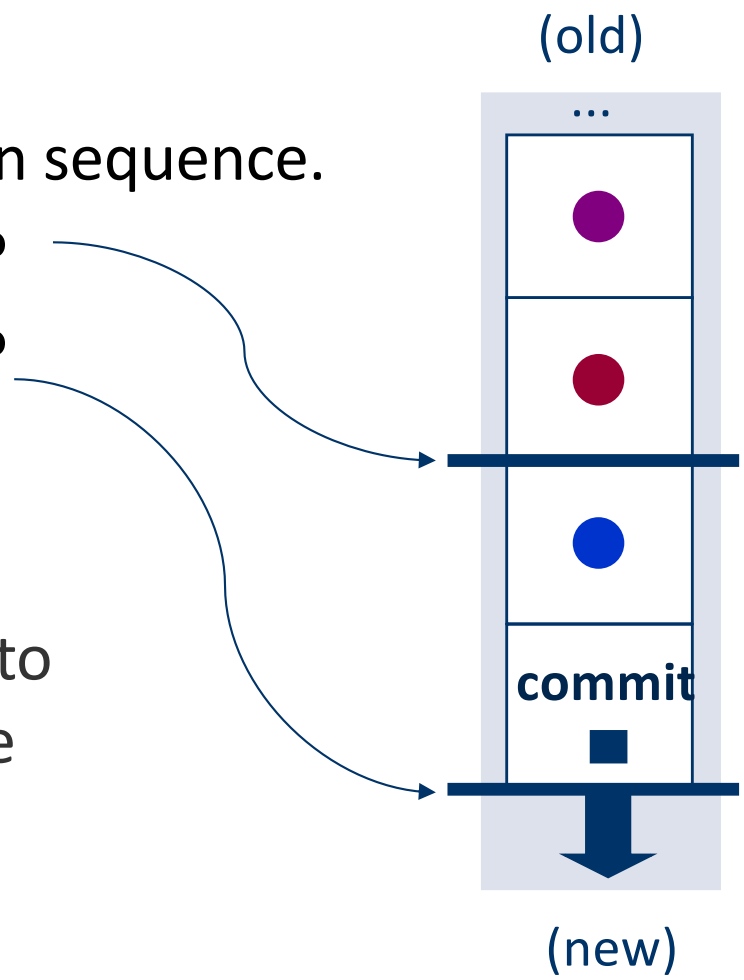
Commit record

- Log entries may represent individual update steps of a transaction.
- **Idea**: log each step, and then log a **commit entry** to record that the op is complete.
- **Assume**: log entries persist "in order" (append).
- **On recovery**: if commit record is present, replay all steps to redo the op, else ignore.



Recovering commits

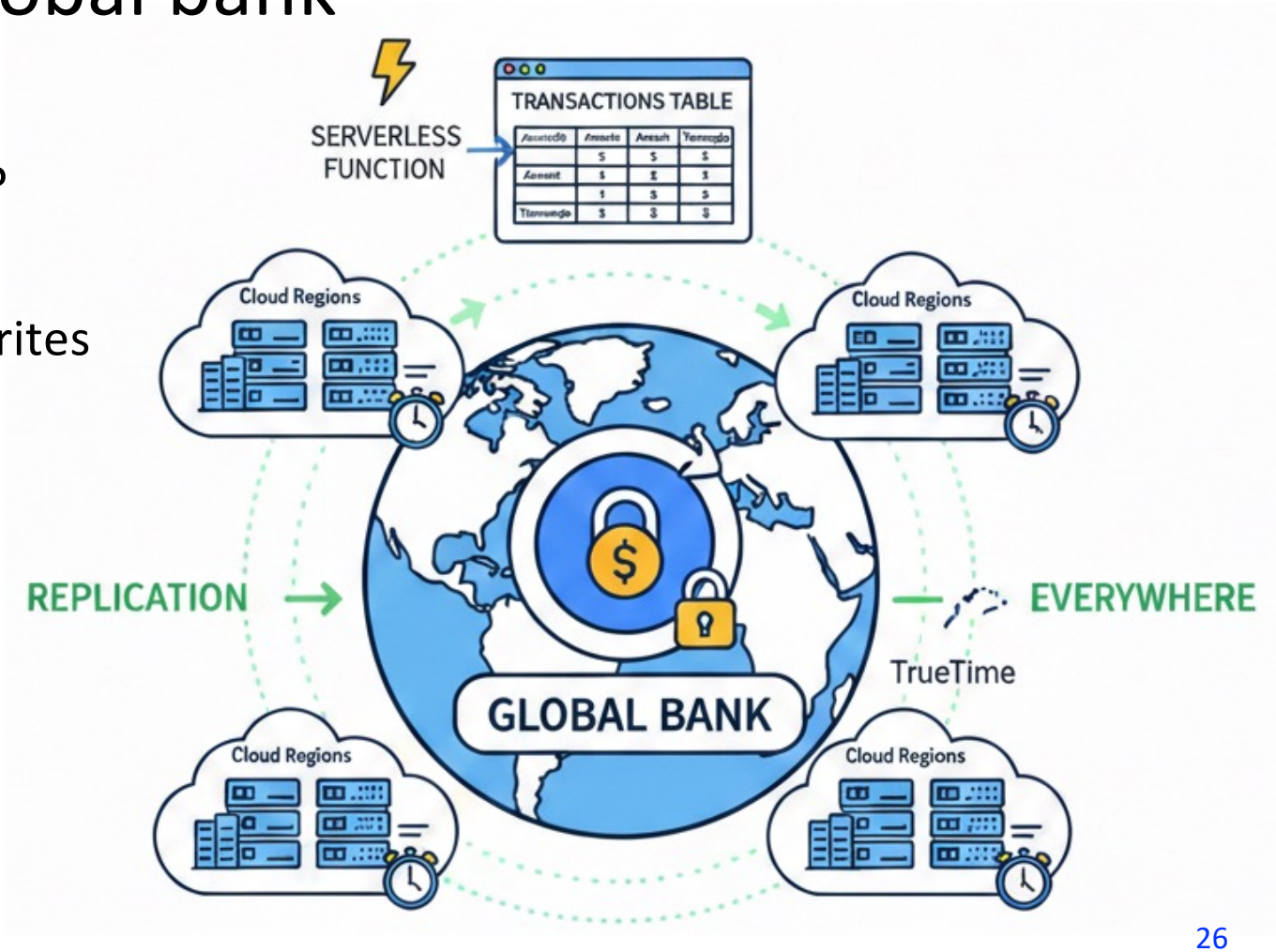
- Imagine logging each entry in sequence.
- If we crash **here**, then what?
- If we crash **here**, then what?



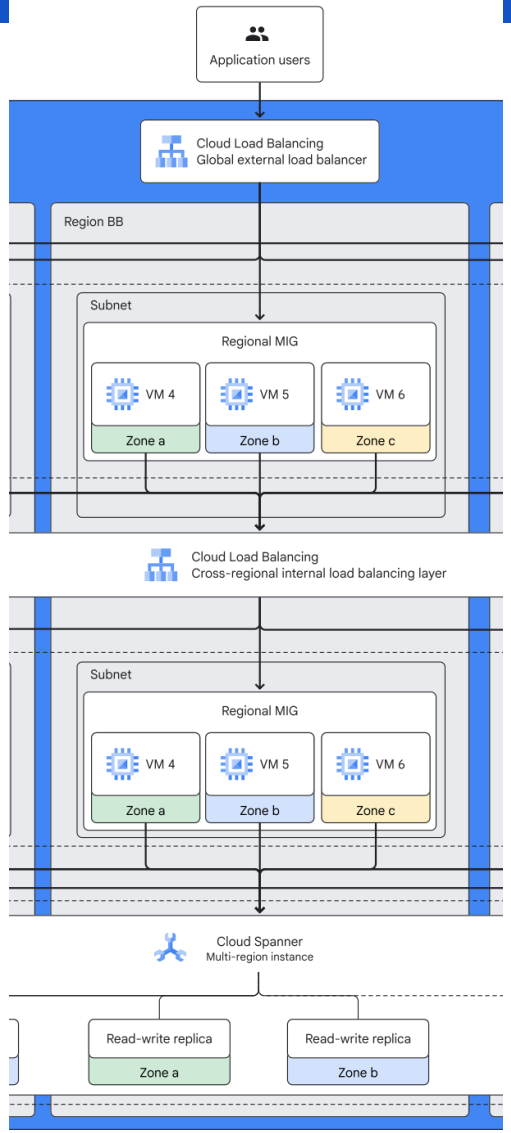
After a failure, replay the log into the last snapshot to recover the data structure.

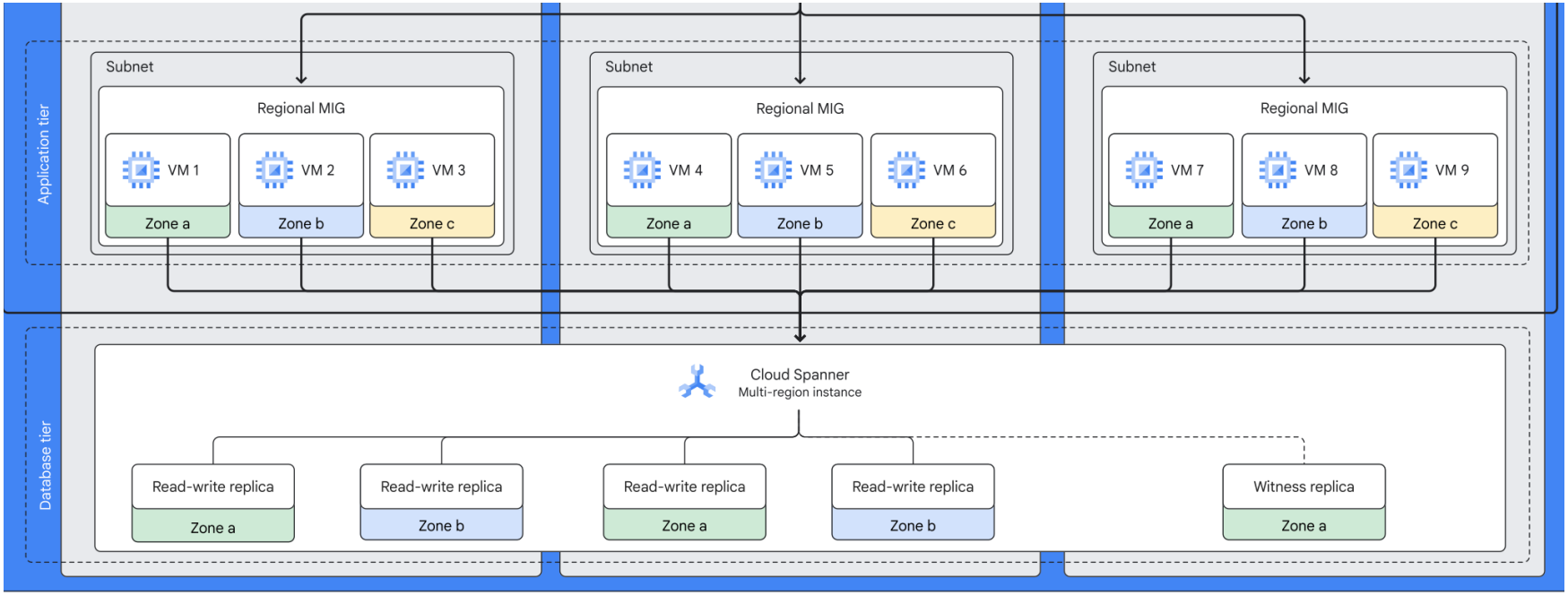
Example app: global bank

- Numbered accounts
- Grouped in branches?
- Sharded
- Complex reads and writes
  - Balance queries
  - Transfers
  - Cross-shard



Gemini drew it


Application users
Cloud Load Balancing
Global external load balancer
Region BB
Subnet
Regional MIG
VM 4
VM 5
VM 6
Zone a
Zone b
Zone c
Cloud Load Balancing
Cross-regional internal load balancing layer
Subnet
Regional MIG
VM 4
VM 5
VM 6
Zone a
Zone b
Zone c
Cloud Spanner
Multi-region instance
Read-write replica
Zone a
Read-write replica
Zone b


Application tier
Subnet
Regional MIG
VM 1
VM 2
VM 3
Zone a
Zone b
Zone c
Subnet
Regional MIG
VM 4
VM 5
VM 6
Zone a
Zone b
Zone c
Subnet
Regional MIG
VM 7
VM 8
VM 9
Zone a
Zone b
Zone c
Database tier
Cloud Spanner
Multi-region instance
Read-write replica
Zone a
Read-write replica
Zone b
Read-write replica
Zone a
Read-write replica
Zone b
Witness replica
Zone a

```
/**
 * Best Practice: Initialize Spanner outside the function handler
 * to reuse sessions across multiple function invocations.
 */
const { Spanner } = require('@google-cloud/spanner');

// Global variables are cached between executions
const spanner = new Spanner({ projectId: 'your-project-id' });
const instance = spanner.instance('your-instance-id');
const database = instance.database('your-database-id');

exports.bankTransfer = async (req, res) => {
  const { fromId, toId, amount } = req.body;

  try {
    // Spanner manages the transaction and session checkout automatically
    await database.runTransactionAsync(async (transaction) => {
      // 1. Get current balances
      const [rowA] = await transaction.run(`SELECT Balance FROM Accounts WHERE Id = ${fromId}`);
      const [rowB] = await transaction.run(`SELECT Balance FROM Accounts WHERE Id = ${toId}`);

      const balanceA = rowA[0].toJSON().Balance;
      const balanceB = rowB[0].toJSON().Balance;

      if (balanceA < amount) throw new Error('Insufficient funds');

      // 2. Perform the transfer
      transaction.update('Accounts', [
        { Id: fromId, Balance: balanceA - amount },
        { Id: toId,   Balance: balanceB + amount }
      ]);

      await transaction.commit();
    });

    res.status(200).send(`Transfer of ${amount} successful.`);
  } catch (err) {
    console.error('Transaction failed:', err);
    res.status(500).send(err.message);
  }
};
```